

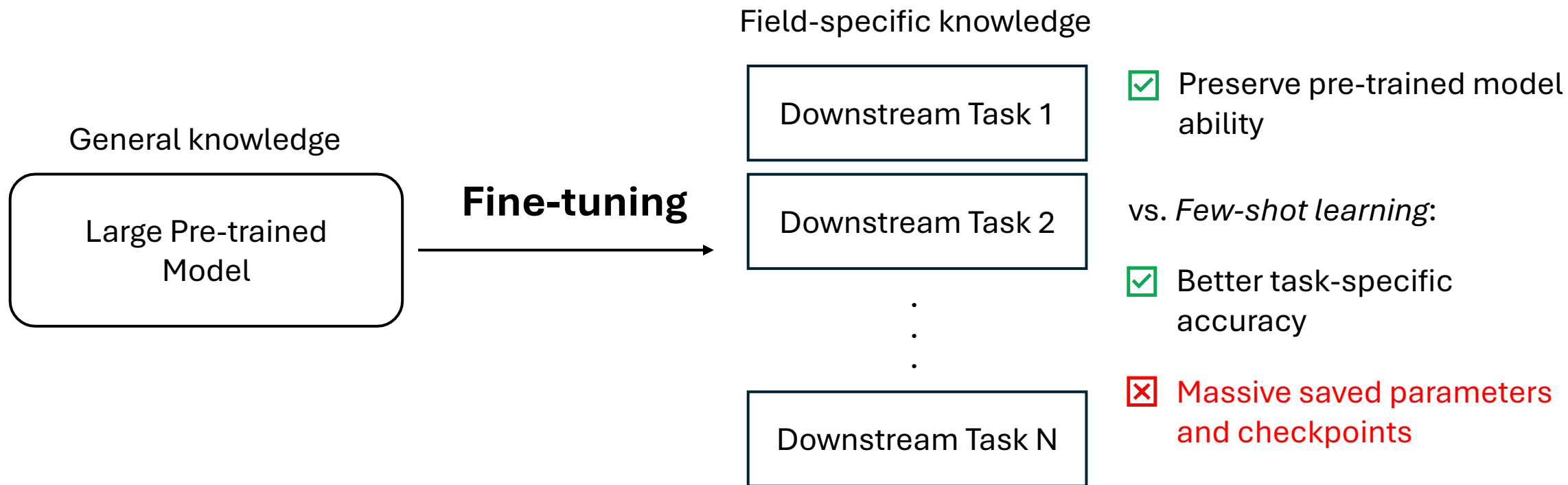
LORA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS

Presenter:

Dingyuan Cao, Hui Ren, Steve Wang, Chang Liu, Yihang Jiao

Motivation

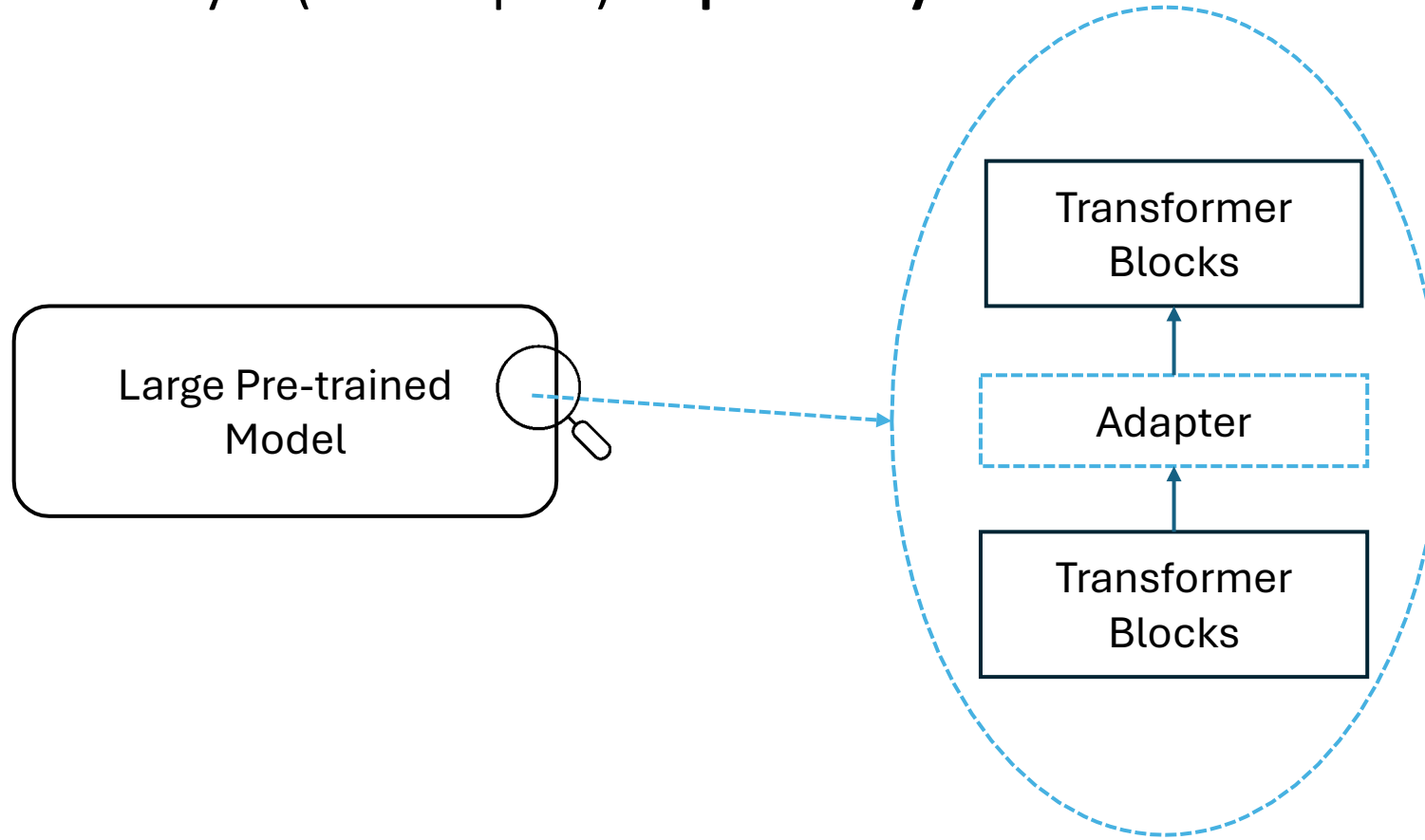
Large Model Finetuning



Motivation

Previous fine-tuning method: adapter-based tuning

- Insert layer (i.e: adapter) **sequentially** between blocks



vs. *Full Fine-tuning*:

✓ Fewer parameters

However, in a ***real system***:

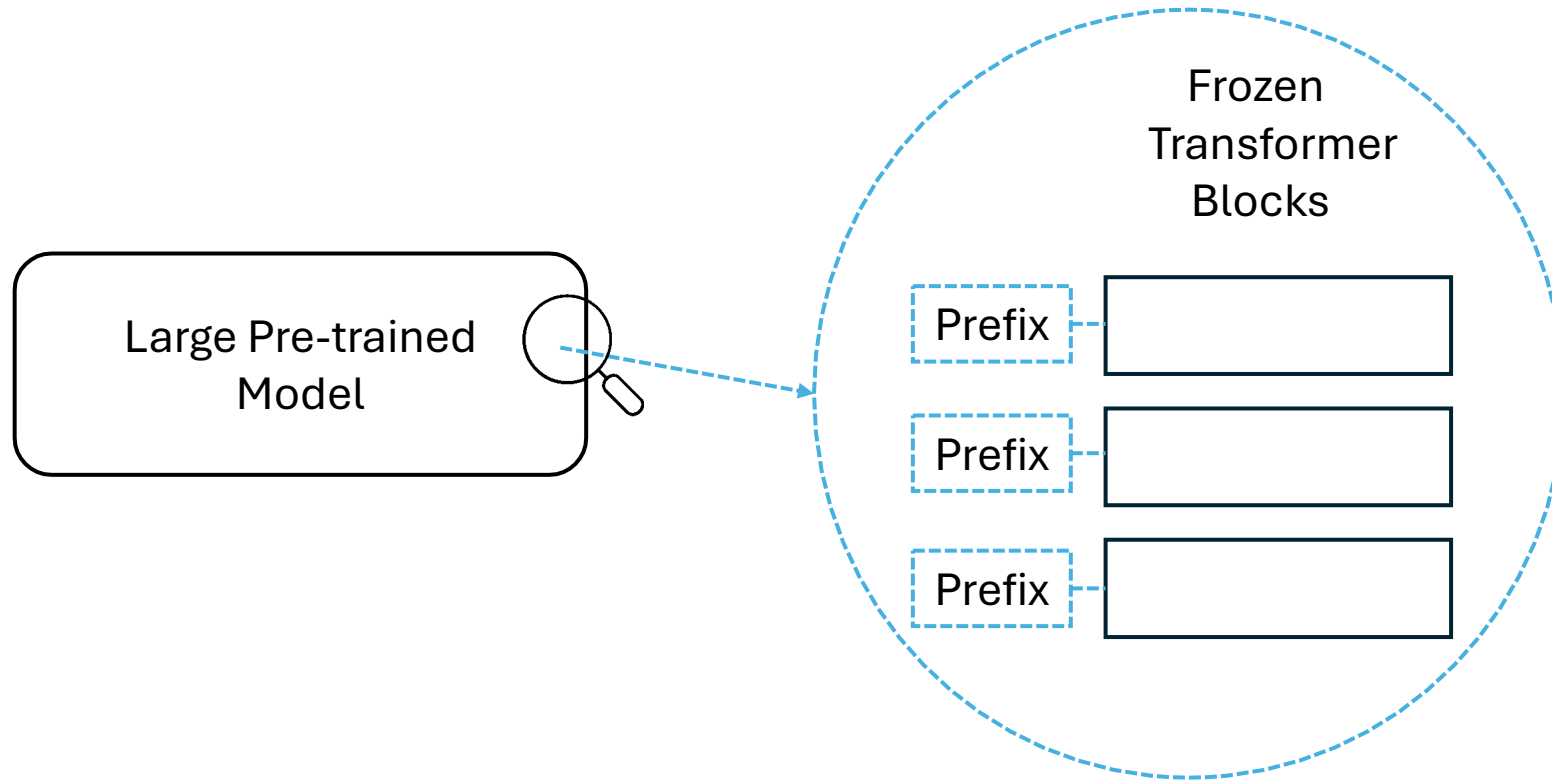
✗ Increase inference latency

✗ Struggle with parallelism

Motivation

Previous fine-tuning method: prefix tuning

- Add “prefix” to input and optimize their embeddings



vs. *Full Fine-tuning*:

- ✓ Even fewer parameters
- ✗ Less tokens left for task
- ✗ Performance not scale linearly with parameter

Main Challenges

Balance between **inference latency vs performance**

Target:

- Reduce number of trainable parameters per task
- Enable efficient inference
- Ensure no performance drop compared to full fine-tuning
- Combine with other fine-tuning methods

Method

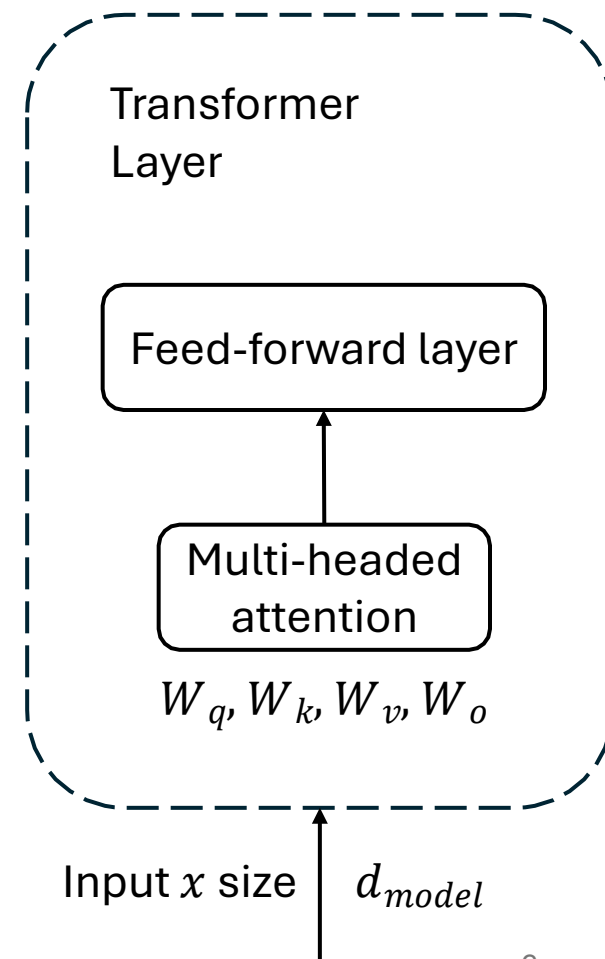
Notations and Problem Statement

Model: Language model

- Structure: transformer based

Weight update

- Pre-trained parameter: W_0 ; Updated parameter: ΔW_0
- $W_0 \rightarrow W_0 + \Delta W_0$
- Previous fine-tuning method: update full rank $\Delta W_0 \in R^{(d \times d)}$



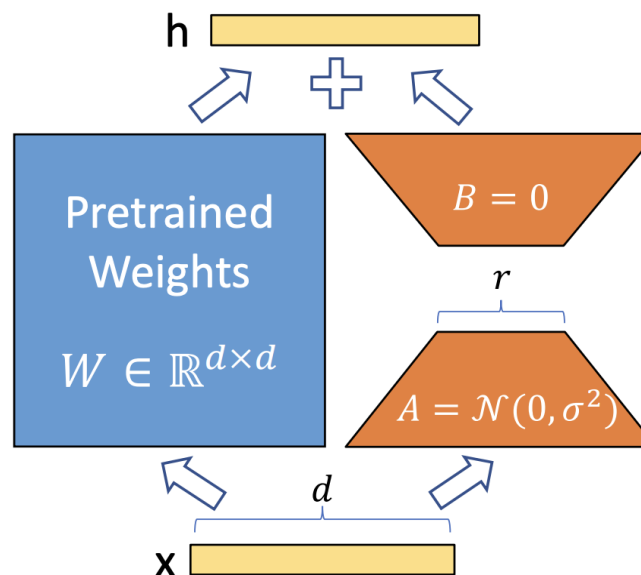
Method

Hypothesis: pre-trained model (in R^d space) could learn on **lower subspace R^r**

Implementation: $\Delta W^{d \times d} = B^{d \times r} \times A^{r \times d}$

- W_0 is frozen
- $r \ll d$

$$h = W_0 x + \Delta W x = W_0 x + B A x$$

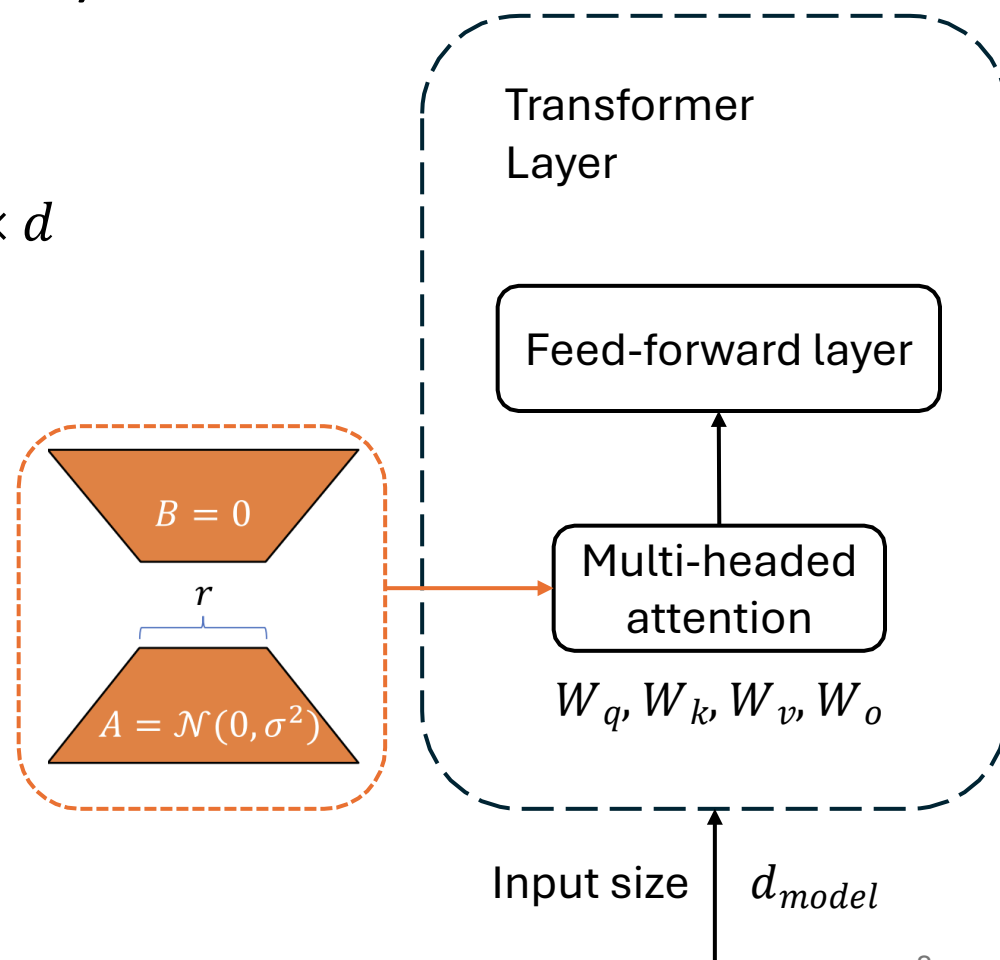


Method

Hypothesis: pre-trained model (in R^d space) could learn on **lower subspace R^r**

Implementation: $\Delta W^{d \times d} = B^{d \times r} \times A^{r \times d}$

- W_0 is frozen
- $r \ll d$



Method

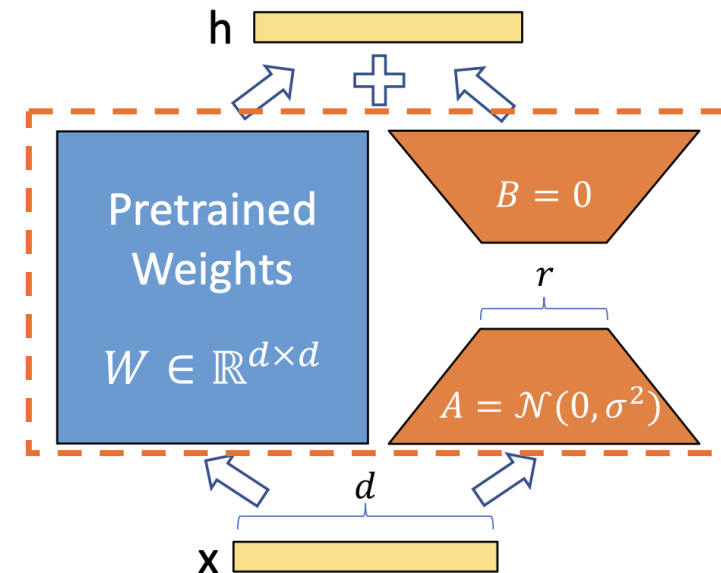
Implementation: $\Delta W^{d \times d} = B^{d \times r} \times A^{r \times d}$

- W_0 is frozen
- $r \ll d$

✓ Fewer trainable parameters: low rank design

Updated parameter size: d^2 vs $2 \times d \times r$

✓ Weight merging : Zero inference latency



Method

Implementation: $\Delta W^{d \times d} = B^{d \times r} \times A^{r \times d}$

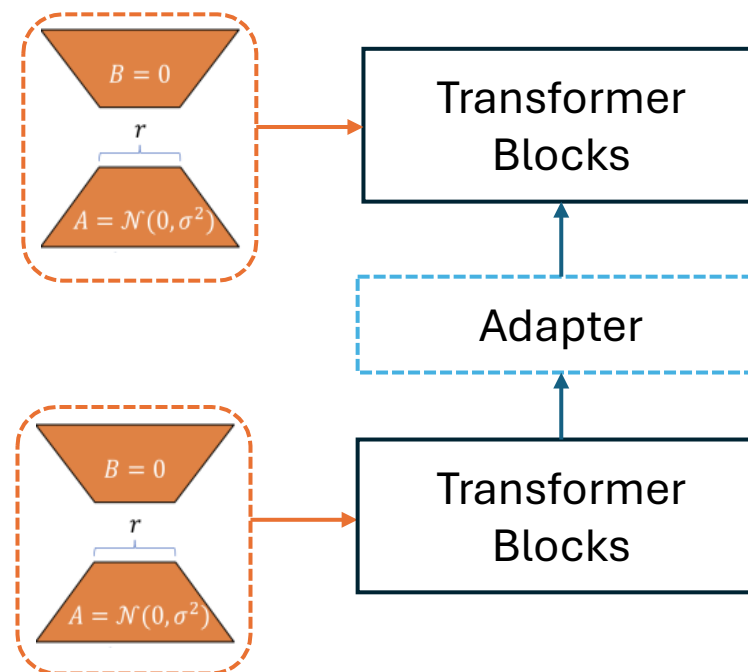
- W_0 is frozen
- $r \ll d$

✓ Fewer trainable parameters: low rank design

Updated parameter size: d^2 vs $2 \times d \times r$

✓ Weight merging : Zero inference latency

✓ Decoupled adaptation: compatible with other fine-tuning methods



Experimental Setup & Baselines-Part1

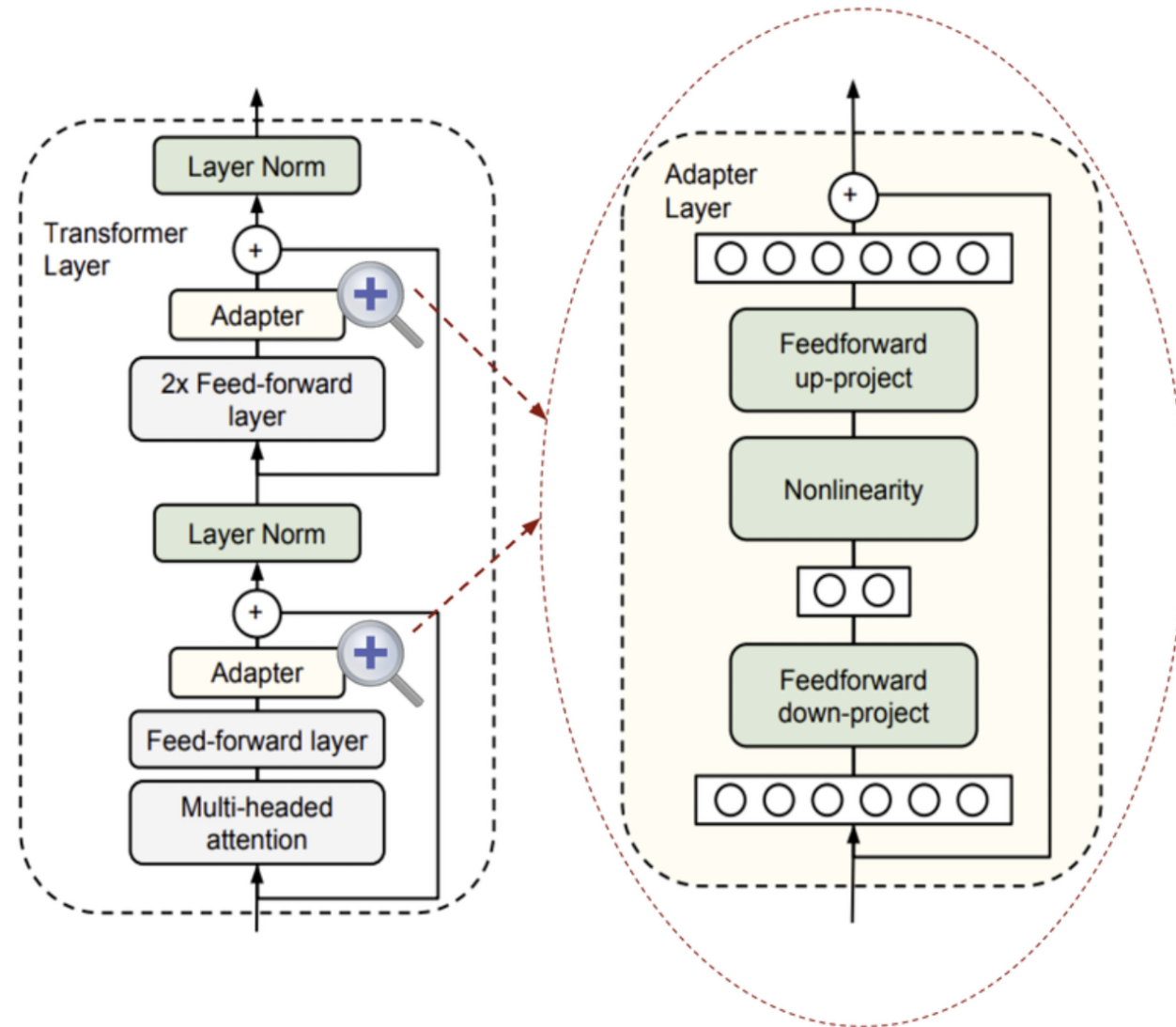
Objective: Compare LoRA broadly against established methods.

Fine-Tuning

- Initialize with pre-trained weights.
- Update all model parameters via gradient descent.
- FTTop2: Simple variant updating only the last two layers.

Bias-only (BitFit):

- Freeze all weights.
- Train **only** the bias vectors.
- Highly parameter-efficient but limited expressiveness.



Experimental Setup & Baselines-Part2

Prefix-based Methods:

- **PreEmbed (Prefix-embedding tuning):** Inserts special trainable tokens (prefix/infix) into input.
Param count: $|\Theta| = d_{model} \times (l_p + l_i)$.
- **PreLayer (Prefix-layer tuning):** PreLayer extends this to every transformer layer, learning full activations.

$$\text{Param count: } |\Theta| = L \times d_{model} \times (l_p + l_i).$$

Adapter Methods:

- Insert adapter layers (bottleneck structure) inside Transformer blocks.
- **AdapterH:** Between Self-Attention and MLP.
- **AdapterL / AdapterP:** Only after MLP and LayerNorm (more efficient).
- **AdapterD:** Drops some adapter layers for efficiency.

L = the number of Transformer layers in the model

d_{model} = the hidden dimension

$l_p + l_i$ = the number of prefix + infix tokens you're inserting

LoRA Parameter Count

- **Implementation:**

- Adds trainable rank decomposition matrices in parallel to existing weights.
- Applied primarily to W_q and W_v (Attention and Value matrices) for simplicity.

- **Parameter Count:**

$$|\Theta| = 2 \times L_{LoRA} \times d_{model} \times r$$

- Dependent on rank r and original weight shape.

L_{LoRA} = the number of weight matrices you apply LoRA to

d_{model} = the hidden dimension / model width

The **2** comes from having two matrices per LoRA module: A ($d \times r$) and B ($r \times d$)

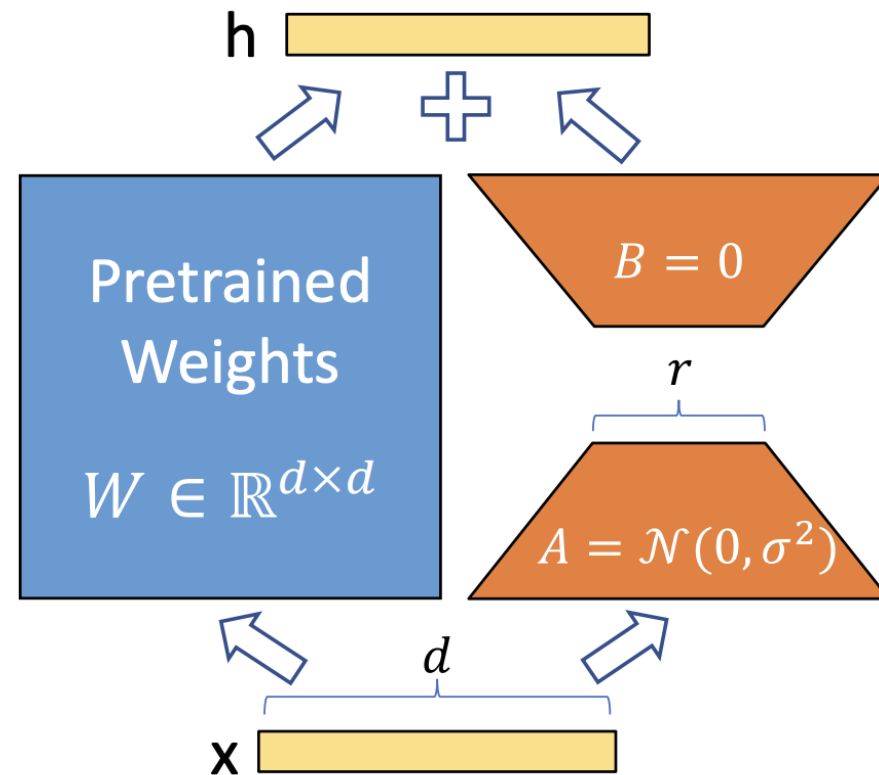


Figure 1: Our reparametrization. We only train A and B .

Key Takeaways:

- **Mechanism:** We freeze the pre-trained weight W (blue) and only train the low-rank matrices A and B (orange).
- **Initialization:** A is Gaussian initialized, B is zero initialized, ensuring $\Delta W = 0$ at the start of training.

Experiment 1 - RoBERTa (Base/Large)

- **Model:** RoBERTa Base (125M) and Large (355M).
- **Benchmark:** GLUE tasks.
- **Strict Comparison Protocol (vs. Adapters):**
 - **Batch Size & Seq Length:** Fixed to match adapter baselines (Seq Len = 128).
 - **Initialization:** No intermediate fine-tuning on MNLI for small tasks (MRPC, RTE, STS-B). Started directly from pre-trained weights.
- **Result:** LoRA achieves competitive performance under these restricted "fair" conditions.

Experiment 2 - Scaling & Generative Tasks

- **DeBERTa XXL (1.5B Parameters):**
 - **Goal:** Test LoRA on large-scale NLU models.
 - **Result:** LoRA matches fully fine-tuned DeBERTa XXL performance on GLUE.
- **GPT-2 Medium/Large (NLG):**
 - **Goal:** Validate LoRA on GPT-2 for Natural Language Generation (E2E NLG Challenge).
 - **Setup:** Aligned with Li & Liang (2021) for direct comparison.
 - **Result:** LoRA holds up on generation tasks, not just classification.

Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB _{base} (FT)*	125.0M	87.6	94.8	90.2	63.6	92.8	91.9	78.7	91.2	86.4
RoB _{base} (BitFit)*	0.1M	84.7	93.7	92.7	62.0	91.8	84.0	81.5	90.8	85.2
RoB _{base} (Adpt ^D)*	0.3M	87.1 $\pm$.0	94.2 $\pm$.1	88.5 $\pm$ 1.1	60.8 $\pm$.4	93.1 $\pm$.1	90.2 $\pm$.0	71.5 $\pm$ 2.7	89.7 $\pm$.3	84.4
RoB _{base} (Adpt ^D)*	0.9M	87.3 $\pm$.1	94.7 $\pm$.3	88.4 $\pm$.1	62.6 $\pm$.9	93.0 $\pm$.2	90.6 $\pm$.0	75.9 $\pm$ 2.2	90.3 $\pm$.1	85.4
RoB _{base} (LoRA)	0.3M	87.5 $\pm$.3	95.1 $\pm$.2	89.7 $\pm$.7	63.4 $\pm$ 1.2	93.3 $\pm$.3	90.8 $\pm$.1	86.6 $\pm$.7	91.5 $\pm$.2	87.2
RoB _{large} (FT)*	355.0M	90.2	96.4	90.9	68.0	94.7	92.2	86.6	92.4	88.9
RoB _{large} (LoRA)	0.8M	90.6 $\pm$.2	96.2 $\pm$.5	90.9 $\pm$ 1.2	68.2 $\pm$ 1.9	94.9 $\pm$.3	91.6 $\pm$.1	87.4 $\pm$ 2.5	92.6 $\pm$.2	89.0
RoB _{large} (Adpt ^P)†	3.0M	90.2 $\pm$.3	96.1 $\pm$.3	90.2 $\pm$.7	68.3 $\pm$ 1.0	94.8 $\pm$.2	91.9 $\pm$.1	83.8 $\pm$ 2.9	92.1 $\pm$.7	88.4
RoB _{large} (Adpt ^P)†	0.8M	90.5 $\pm$.3	96.6 $\pm$.2	89.7 $\pm$ 1.2	67.8 $\pm$ 2.5	94.8 $\pm$.3	91.7 $\pm$.2	80.1 $\pm$ 2.9	91.9 $\pm$.4	87.9
RoB _{large} (Adpt ^H)†	6.0M	89.9 $\pm$.5	96.2 $\pm$.3	88.7 $\pm$ 2.9	66.5 $\pm$ 4.4	94.7 $\pm$.2	92.1 $\pm$.1	83.4 $\pm$ 1.1	91.0 $\pm$ 1.7	87.8
RoB _{large} (Adpt ^H)†	0.8M	90.3 $\pm$.3	96.3 $\pm$.5	87.7 $\pm$ 1.7	66.3 $\pm$ 2.0	94.7 $\pm$.2	91.5 $\pm$.1	72.9 $\pm$ 2.9	91.5 $\pm$.5	86.4
RoB _{large} (LoRA)†	0.8M	90.6 $\pm$.2	96.2 $\pm$.5	90.2 $\pm$ 1.0	68.2 $\pm$ 1.9	94.8 $\pm$.3	91.6 $\pm$.2	85.2 $\pm$ 1.1	92.3 $\pm$.5	88.6
DeB _{XXL} (FT)*	1500.0M	91.8	97.2	92.0	72.0	96.0	92.7	93.9	92.9	91.1
DeB _{XXL} (LoRA)	4.7M	91.9 $\pm$.2	96.9 $\pm$.2	92.6 $\pm$.6	72.4 $\pm$ 1.1	96.0 $\pm$.1	92.9 $\pm$.1	94.9 $\pm$.4	93.0 $\pm$.2	91.3

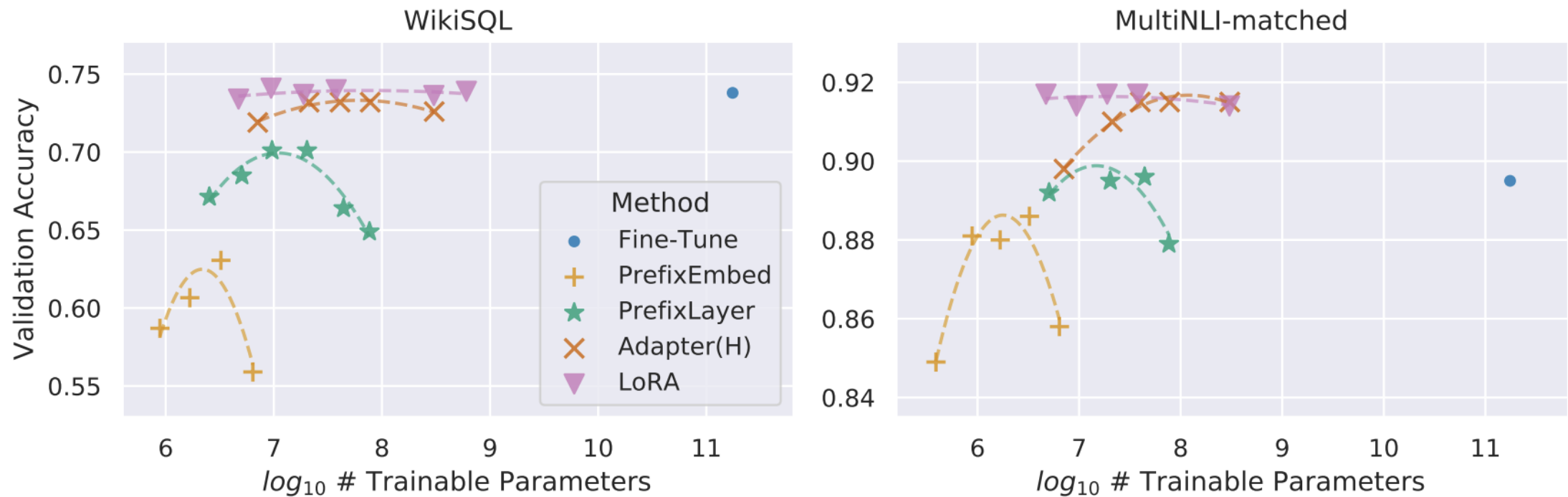
Performance Comparison on GLUE Benchmark: LoRA vs. Fine-Tuning (FT) and Adapters.

Key Takeaways :

- **Competitive Performance:** LoRA with only 0.8M trainable parameters matches or beats full fine-tuning at 355M parameters on RoBERTa Large. On DeBERTa XXL, LoRA with 4.7M parameters matches full fine-tuning at 1,500M.
- **Outperforming Adapters:** LoRA consistently outperforms Adapter-based methods (AdapterH, AdapterL, AdapterP) and Bias-only (BitFit), even when they use a similar or larger parameter budget.
- The dagger rows show this holds even under the strict fair-comparison setup.
- The results are clear: LoRA is not just parameter-efficient, it's genuinely competitive on quality.

Experiment 3 - The Stress Test (GPT-3 175B)

- **Scale:** 175 Billion parameters.
- **Performance:** LoRA matches or exceeds the fine-tuning baseline on all tested datasets.
- **The "More Parameters" Trap:**
 - Prefix-tuning performance **drops** significantly when using >256 special tokens.
 - **Hypothesis:** Too many special tokens shift input distribution away from pre-training data. LoRA has no such issue.



Model Performance vs. Number of Trainable Parameters on GPT-3 175B.

Key Takeaways :

- **Monotonic Improvement:** LoRA's performance improves or remains stable as we increase the rank r (more parameters).
- **Competitor's Failure Mode:** Prefix-tuning suffers a **significant performance drop** when using too many special tokens (>256).
- **Reason:** Excessive prefix tokens likely cause the input distribution to shift too far from the pre-training data, whereas LoRA avoids this issue.

Which Weights to Adapt? (Budget Allocation)

- **Research Question:** Given a fixed parameter budget like 18M params, how to allocate resources?
- **Comparison:**
 - **Strategy A:** Adapt one matrix type like ΔW_q with larger r ($r=8$).
 - **Strategy B:** Adapt multiple types (ΔW_q & ΔW_v) with smaller r ($r=4$).
- **Finding:** Adapting both W_q and W_v yields better results.
- **Conclusion:** $r=4$ captures sufficient information. It is preferable to adapt more weight types than to increase rank on a single type.

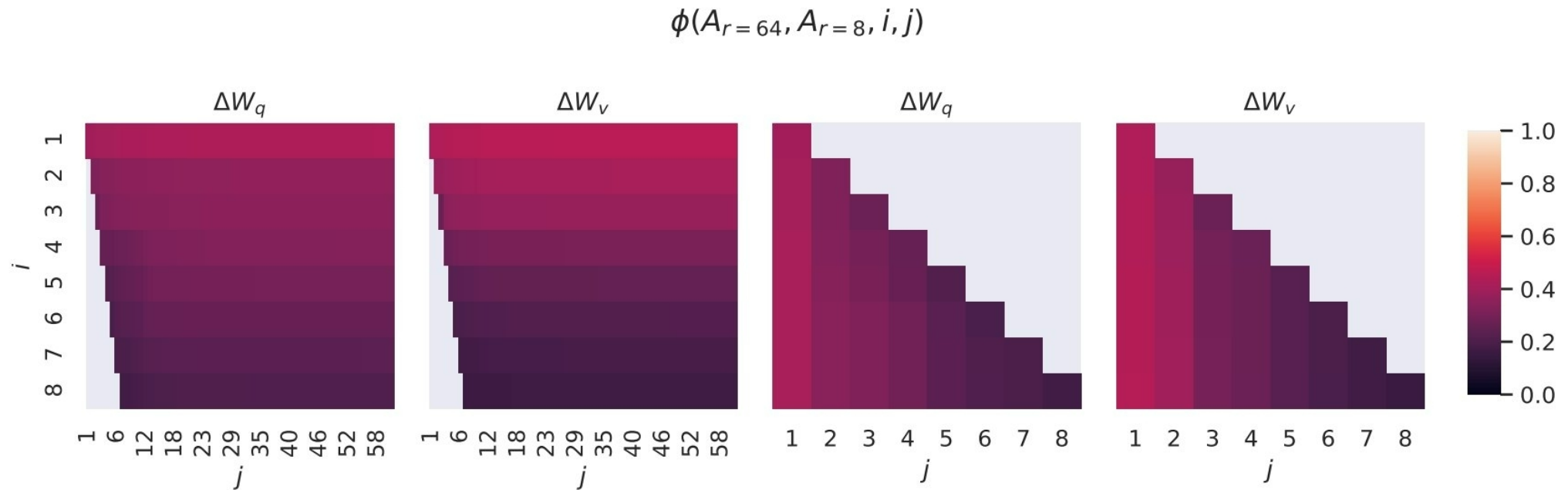
	# of Trainable Parameters = 18M						
Weight Type Rank r	W_q 8	W_k 8	W_v 8	W_o 8	W_q, W_k 4	W_q, W_v 4	W_q, W_k, W_v, W_o 2
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	73.7	73.7
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	91.7

Validation Accuracy on GPT-3 175B: Allocating a Fixed Parameter Budget (18M) Across Different Weight Matrices.

- The table confirms this quantitatively. Looking at WikiSQL, W_q alone at $r=8$ gets 70.4, while W_q and W_v together at $r=4$ jumps to 73.7. Even adding all four attention matrices at $r=2$ ties that score.
- The breadth-vs-depth tradeoff clearly favors breadth. This is a practical design guideline for anyone applying LoRA.

What is the Optimal Rank r ?

- **Observation:** LoRA performs competitively with extremely small ranks like $r=1$ or $r=2$.
- **Subspace Similarity Analysis:**
 - Comparing subspaces learned by $r=8$ vs $r=64$.
 - The top singular directions overlap significantly between the two.
 - Remaining directions in high-rank matrices $r=64$ largely contain random noise.
- **Conclusion:** This suggests the weight update has a very low "intrinsic rank."



Normalized Subspace Similarity between Singular Vectors of $r=8$ and $r=64$.

- This heatmap makes it concrete.
- The dark, high-similarity region is concentrated in the top-left corner, meaning just the top 1 or 2 singular directions are shared and meaningful across different ranks.
- The lighter areas representing higher-order directions show near-zero similarity. For both ΔW_q and ΔW_v , this pattern holds.
- Conclusion: $r=1$ or 2 is often sufficient for effective adaptation.

Relationship Between ΔW and W

- **Question:** Does LoRA learn new features or amplify existing ones?
- **Correlation Analysis:**
 - ΔW correlates stronger with W than random matrices.
 - ΔW does **not** repeat the top singular directions of W .
 - **Key Finding:** ΔW amplifies directions that were **not emphasized** in W (weak features). In other words, LoRA is surfacing task-relevant features that were latent in the pretrained model but not dominant.
- **Amplification Factor:** Huge amplification (approx. 20x) for these specific task-relevant features.

Strength-LoRA Design

- **The Core Intuition: Fine-tuning Doesn't Need Full Rank**

- Pre-trained models already encode rich general knowledge in W_0
- Adapting to a downstream task only requires shifting a small subspace
- Full fine-tuning updates all $d \times k$ parameters, most of it redundant

Why Low-Rank Works: The Intrinsic Dimensionality Argument

- ΔW learned from downstream tasks is empirically rank-deficient
- $r = 1$ or 2 captures the most task-relevant singular directions
- Higher ranks, $r = 64$, add noise, not signal, subspace similarity analysis confirms this

W_0 is the original pretrained weight matrix

$d \times k$ are the dimensions of a weight matrix.

ΔW (Delta W) This is the *change* applied to the pretrained weight.

$r=1$ or 2 / singular directions Singular directions (from SVD — Singular Value Decomposition) are the "principal directions" of a matrix, ranked by importance.

Strength-LoRA Design

- Key Design Choices that Make It Work
 - $B = 0$ at init $\rightarrow \Delta W = 0$, preserving pretrained behavior from step one
 - Parallel path $\rightarrow$ mergeable at inference, no sequential bottleneck
 - α/r scaling $\rightarrow$ stable hyperparameter tuning across different rank choices

B initialized to zero At the very start of training, every value in B is set to 0. This means $B \times A = 0$, so $\Delta W = 0$.

BA (the merged product) At inference time, instead of computing $W_0x + BAx$ separately on every input, you can pre-compute $W' = W_0 + BA$ once, and then just run $W'x$.

α/r scaling factor When you compute the output, LoRA doesn't just add BAx directly. It scales it: $(\alpha/r) \times BAx$. Here:

- α (alpha) is a fixed constant you set as a hyperparameter (e.g., $\alpha=16$)
- r is the rank you chose

Strength-LoRA Experiment

- **Zero Inference Latency:** LoRA matrices BA are merged into W_0 at deployment, no extra computation compared to the base model, unlike adapter layers which add sequential overhead.
- **Extreme Parameter Efficiency Without Quality Loss:**
 - RoBERTa Large: 0.8M trainable params matches full fine-tuning at 355M.
 - DeBERTa XXL: 4.7M params matches full fine-tuning at 1,500M.
 - GPT-3 175B: ~18M params matches or exceeds fine-tuning baseline.
- **Interpretability, What ΔW Actually Does:**
 - ΔW does **not** repeat W 's dominant singular directions.
 - Instead, it amplifies the underemphasized, task-relevant ones with a $\sim 20\times$ factor.
 - **Conclusion:** LoRA isn't learning from scratch. It's surfacing what the pretrained model already knows but hasn't been asked to emphasize. That's why even $r=1$ or 2 is often sufficient.

Summary

Designed for: efficient fine-tuning

Idea: low rank decomposition of weight updates

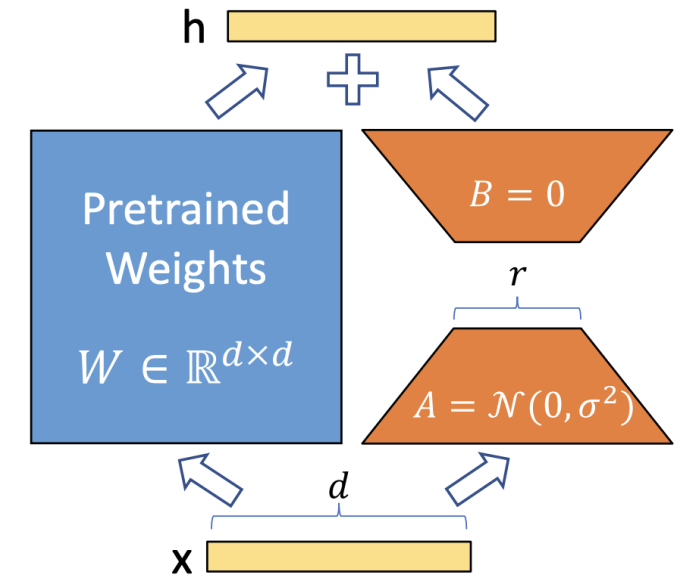
- $\Delta W^{d \times d} = B^{d \times r} \times A^{r \times d} (r \ll d)$

Advantage

- Efficiency
- Zero inference latency

Future Work

- Better allocation of LoRA (Target modules, rank, learning strategies ...)
- Orthogonal design with other fine-tuning and model compression techniques
- Application on other large models



QLoRA: LoRA on a 4-bit base model

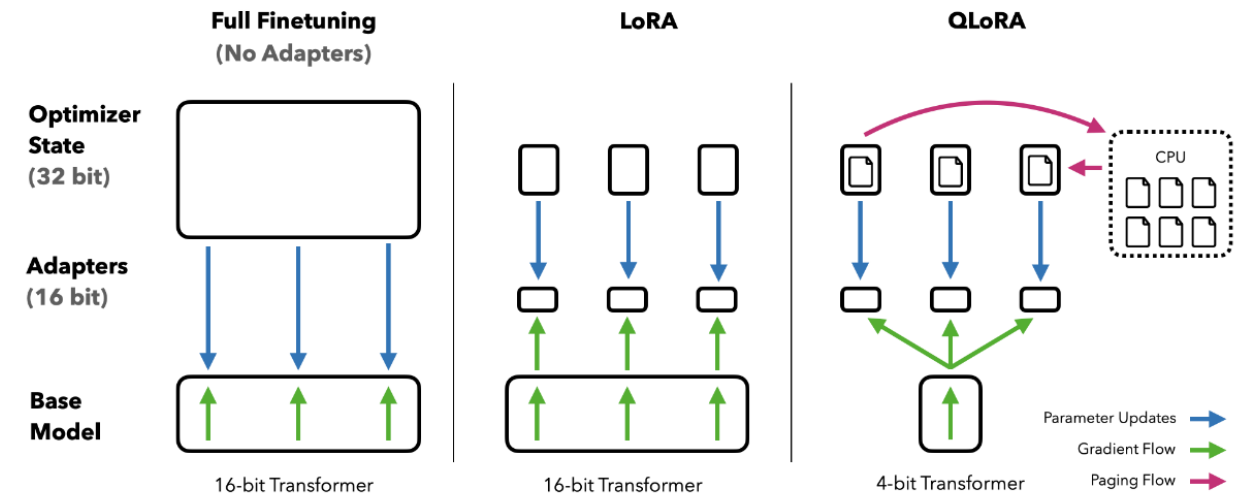
Key idea

- Quantize the frozen base model to 4-bit
- Backprop through quantized weights into LoRA adapters
- Keep optimizer + LoRA weights in higher precision

Practical ingredients

- NF4: a 4-bit data type designed for normally-distributed weights
- Double quantization to reduce quantization constants memory
- Paged optimizers to avoid memory spikes (e.g., with checkpointing)

QLoRA pipeline



Result: memory footprint low enough to fine-tune very large models on a single GPU (paper reports 65B on a 48GB GPU).

AdaLoRA: adapt the rank budget during training

Problem

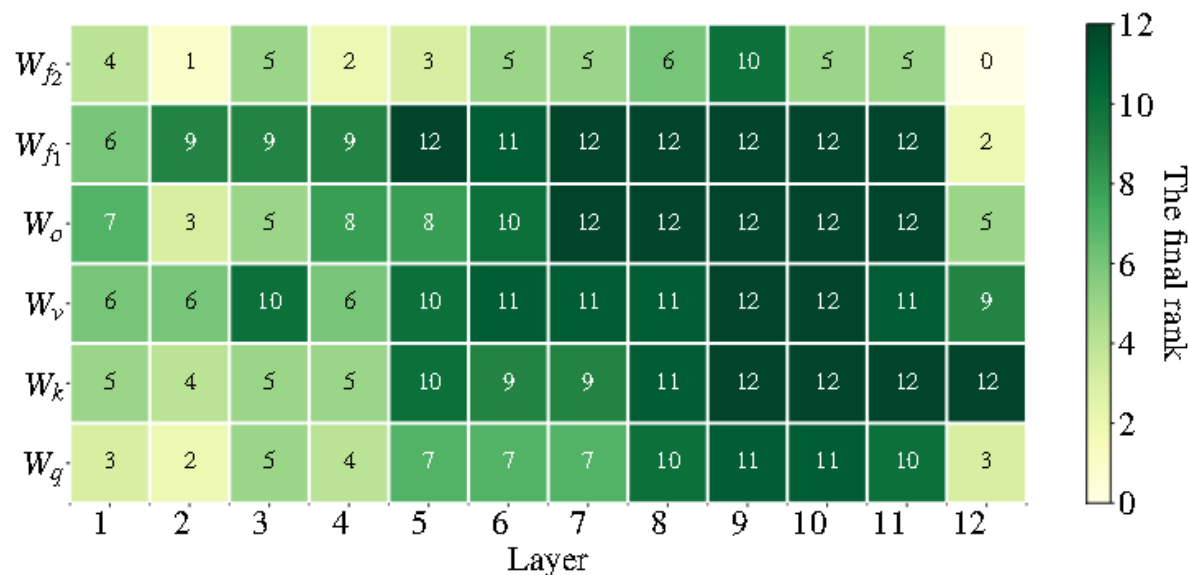
- Standard LoRA uses the same rank everywhere
- But layers / modules vary in importance per task
- Uniform budgets can waste parameters

AdaLoRA approach

- Parameterize ΔW like a lightweight SVD
- Use an importance score to prune unimportant singular values
- Allocate higher rank to more critical modules

$$W = W^{(0)} + \Delta = W^{(0)} + P\Lambda Q,$$

$$S_{k,i} = s(\lambda_{k,i}) + \frac{1}{d_1} \sum_{j=1}^{d_1} s(P_{k,ji}) + \frac{1}{d_2} \sum_{j=1}^{d_2} s(Q_{k,ij}),$$



AdaLoRA targets better quality under the same total adapter budget by reallocating rank during training.

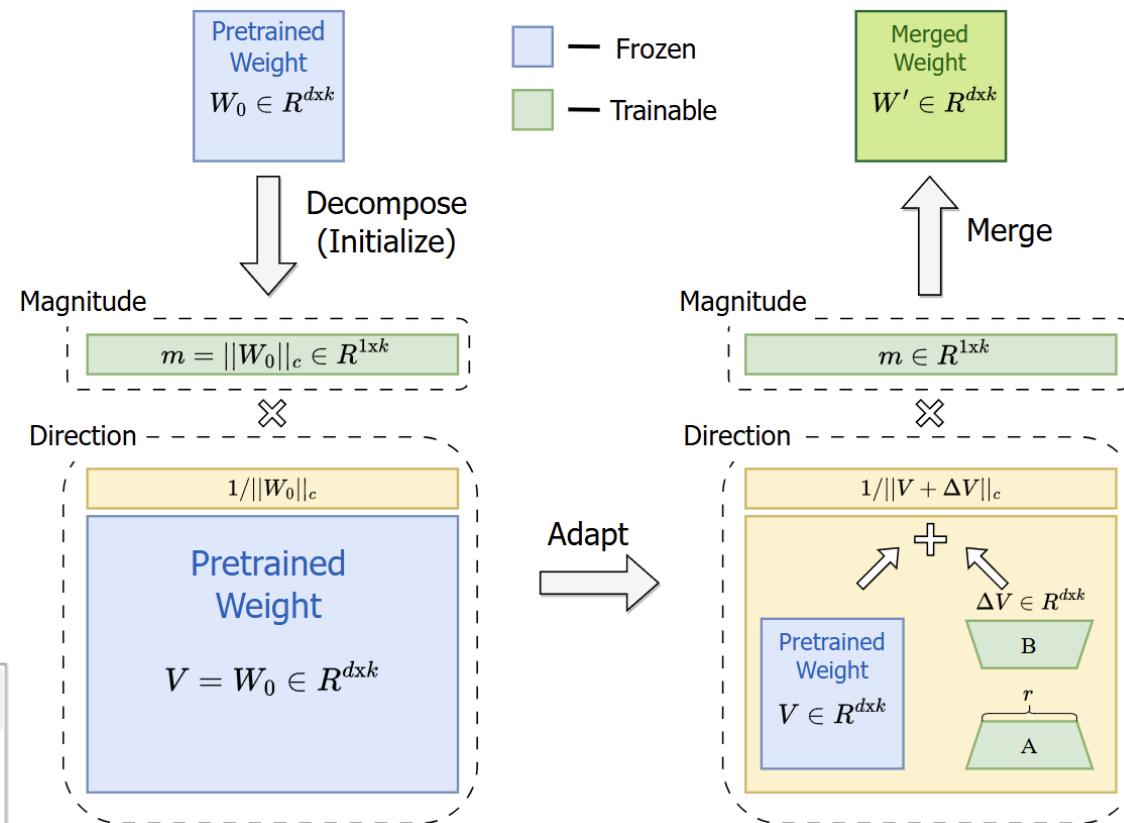
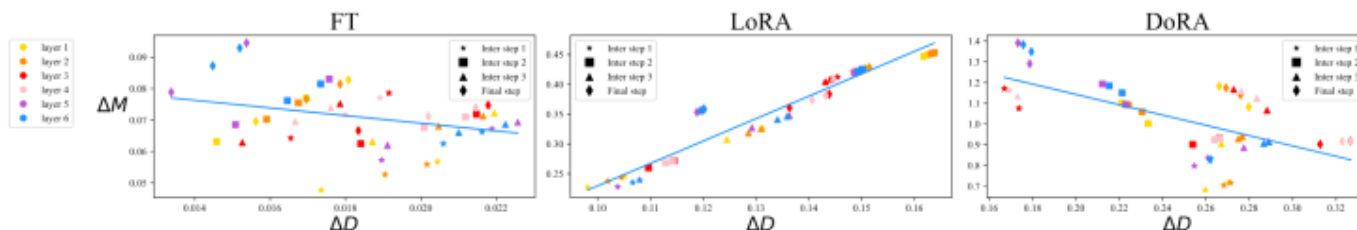
DoRA: decompose weights into magnitude + direction

Motivation

- LoRA can lag full fine-tuning on some tasks
- Full FT changes both “direction” and “scale” of weights
- LoRA updates may not capture this as effectively

DoRA idea

- Decompose weight into magnitude and direction
- Apply LoRA to the directional component
- Learn magnitude separately → closer to full FT capacity



Claim: DoRA improves learning capacity & stability over LoRA while keeping inference cost unchanged.

Where LoRA shows up in practice

LLMs & instruction tuning

- Domain adaptation (legal, medical, finance)
- Instruction / style tuning with small datasets
- Customer-specific personalization without cloning the base model

Multimodal models

- Vision-language instruction tuning (e.g., LLaVA-style)
- Video-text / audio-text adaptation with limited compute

Generative vision (diffusion)

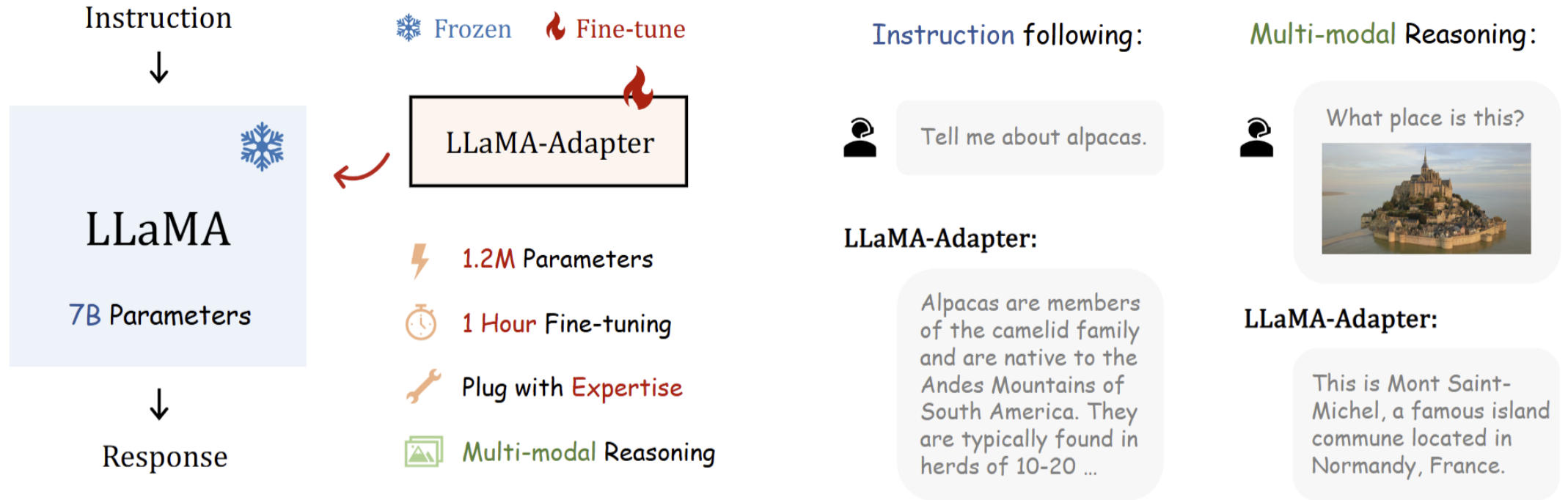
- Compact “style/character” adapters for Stable Diffusion-like models
- Easy community sharing: small files, quick swapping

Serving & lifecycle

- A/B testing by swapping adapters
- Continual learning with adapter libraries
- Edge / on-prem: ship one base model plus tiny deltas

Application: LLMs & instruction tuning

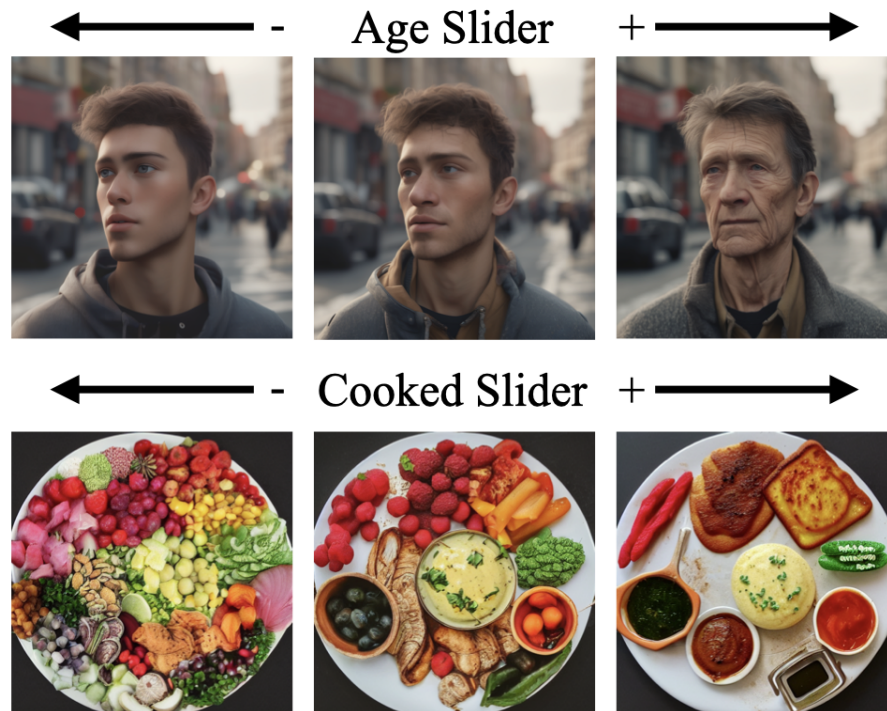
[\[2303.16199\] LLaMA-Adapter: Efficient Fine-tuning of Language Models with Zero-init Attention](#)



Application: Generative vision (diffusion)

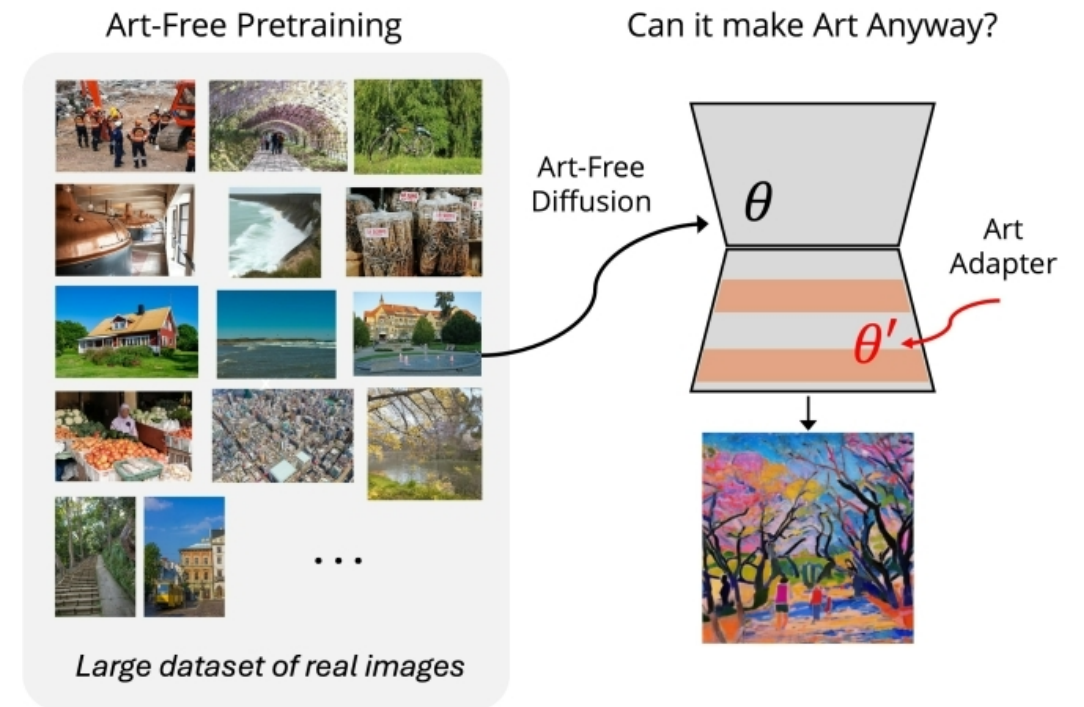
Controllable generation

[\[2311.12092\] Concept Sliders: LoRA Adaptors for Precise Control in Diffusion Models](#)



Few-shot learning

[\[2412.00176\] Opt-In Art: Learning Art Styles Only from Few Examples](#)



dLoRA: Dynamically Orchestrating Requests and Adapters for LoRA LLM Serving

Bingyang Wu et al.

OSDI 2024

Motivation

Low-rank Adaptation(LoRA) is very popular in LLM fine-tuning

- It enables low-cost fine-tuning (~**10,000x** less parameter changed)
- No extra inference cost introduced (LoRA fine-tuned weights are merged)

However, most LLM serving engines do not support LoRA efficiently

- **Orca, vLLM** focus on serving single model
 - Each LoRA + base model is considered a single model, although the base model might be shared
- **PEFT** batches requests for each LoRA adaptor
 - Frequent (un-)merging between batches, low batch size in each batch
- **SHEPHERD, AlpaServe**: DNN serving orchestrator
 - Does not consider the unique characteristics of LoRA

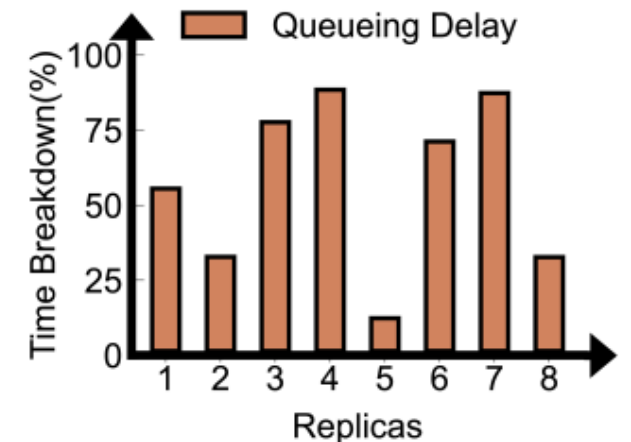
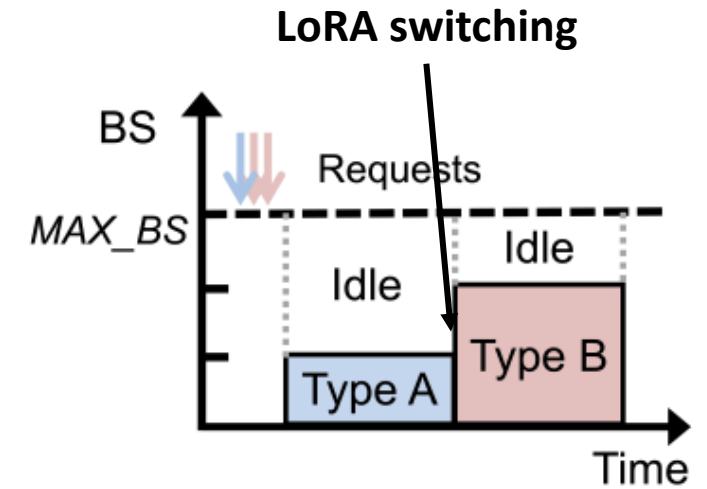
Main Challenges

Challenge 1: Underutilization within a replica

- Consider max batch size 3, and 3 requests coming in
- Due to LoRA switching, it takes 2 time slots with partially filled batch size
- Ideally, we want **1 time slot with filled batch size (3)**

Challenge 2: Load imbalance across replicas

- Due to GPU memory limitation, adapters only lives in a subset of replicas
- Bursty requests/unpredictable output length might overflow replicas
- Need **optimized adaptor scheduling** across replicas

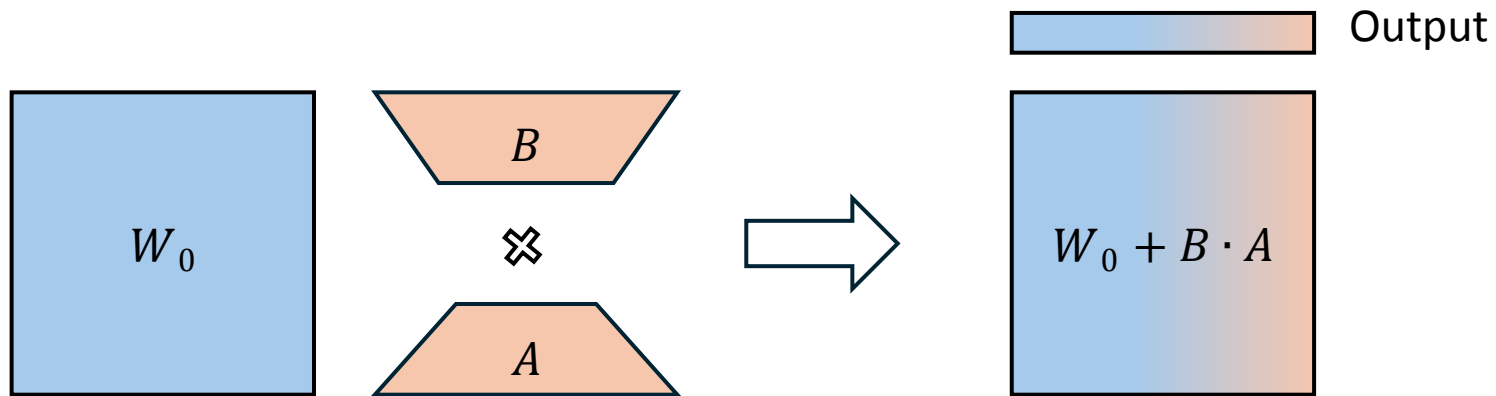


Merged vs. Unmerged Inference

Assume base model weight W_0 , LoRA matrices A, B

The Default: **Merged Inference**

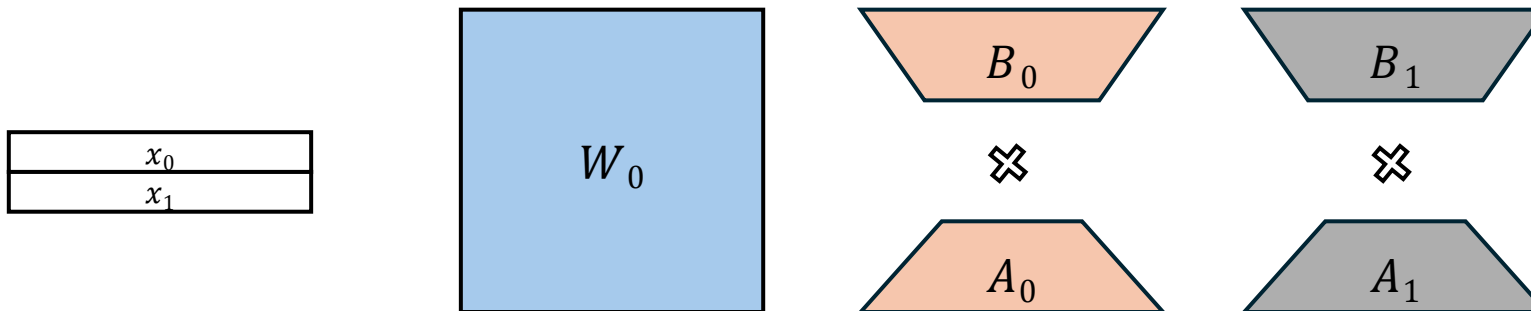
- We first get fine-tuned weight matrix $W = W_0 + B \cdot A$
- In inference, we directly compute Wx
 - **Pros:** no additional inference cost compared to base model (W_0x)
 - **Cons:** need to compute $W_0 + B \cdot A$ each time switching LoRA



Merged vs. Unmerged Inference

Proposed: Unmerged Inference

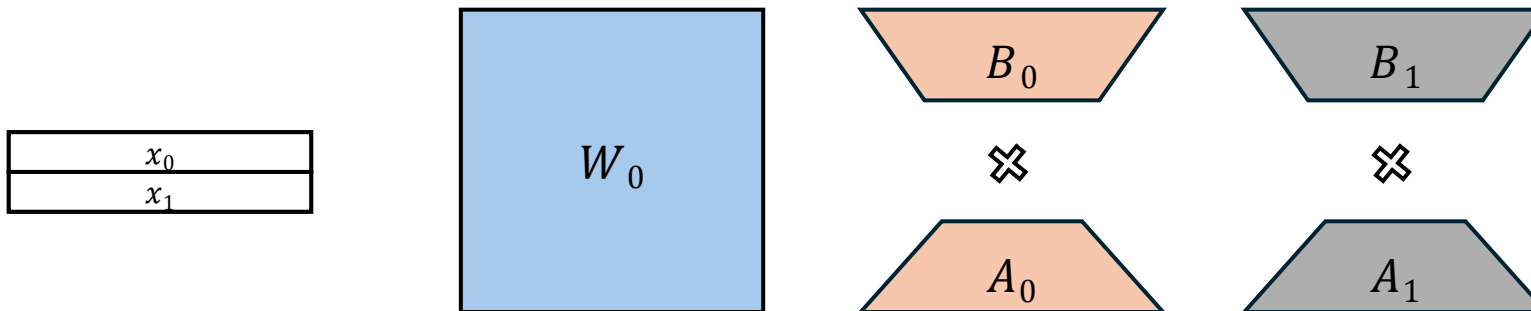
- Consider $x = [x_0, x_1]$, x_0 requires A_0, B_0 , x_1 requires A_1, B_1
- We want $y_0 = W_0x_0 + B_0A_0x_0$, $y_1 = W_0x_1 + B_1A_1x_1$
- Compute $W_0x = [W_0x_0, W_0x_1]$, $B_0A_0x_0$, $B_1A_1x_1$ separately



Merged vs. Unmerged Inference

Proposed: Unmerged Inference

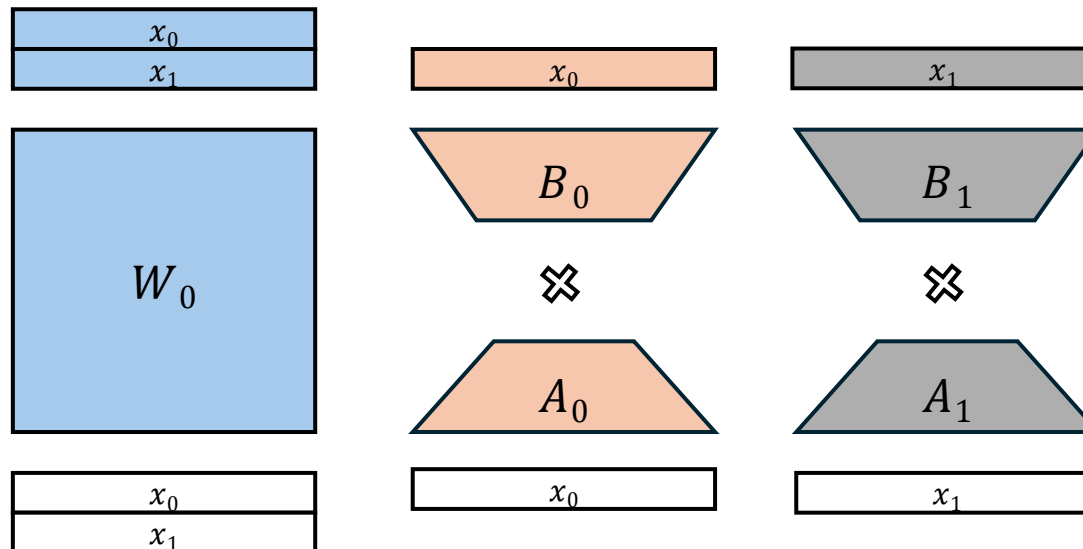
- Consider $x = [x_0, x_1]$, x_0 requires A_0, B_0 , x_1 requires A_1, B_1
- We want $y_0 = W_0x_0 + B_0A_0x_0$, $y_1 = W_0x_1 + B_1A_1x_1$
- Compute $W_0x = [W_0x_0, W_0x_1]$, $B_0A_0x_0$, $B_1A_1x_1$ separately



Merged vs. Unmerged Inference

Proposed: Unmerged Inference

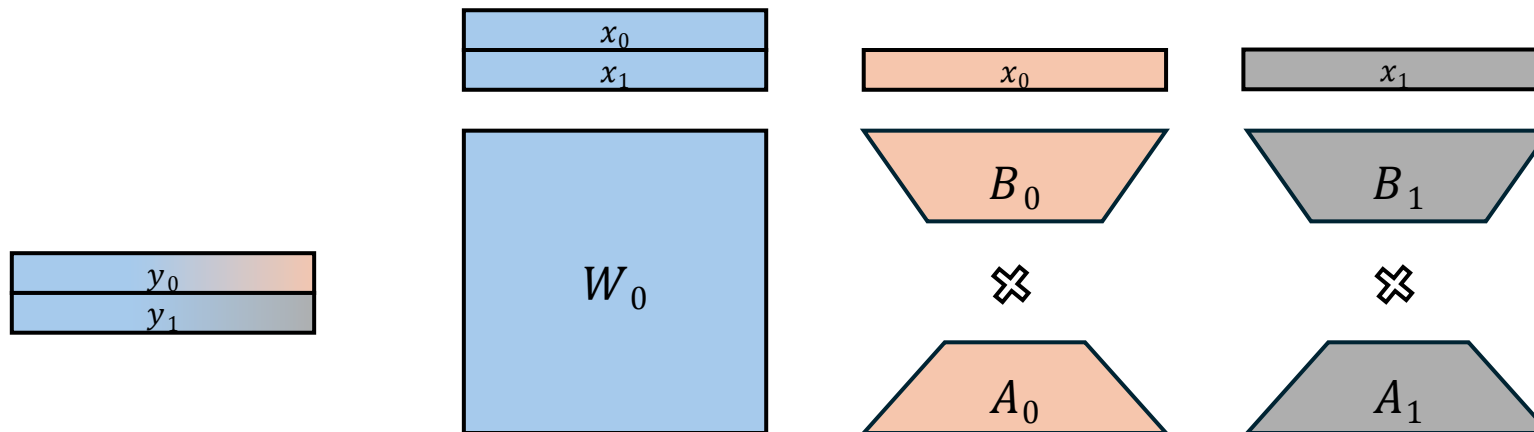
- Consider $x = [x_0, x_1]$, x_0 requires A_0, B_0 , x_1 requires A_1, B_1
- We want $y_0 = W_0x_0 + B_0A_0x_0$, $y_1 = W_0x_1 + B_1A_1x_1$
- Compute $W_0x = [W_0x_0, W_0x_1]$, $B_0A_0x_0$, $B_1A_1x_1$ separately



Merged vs. Unmerged Inference

Proposed: Unmerged Inference

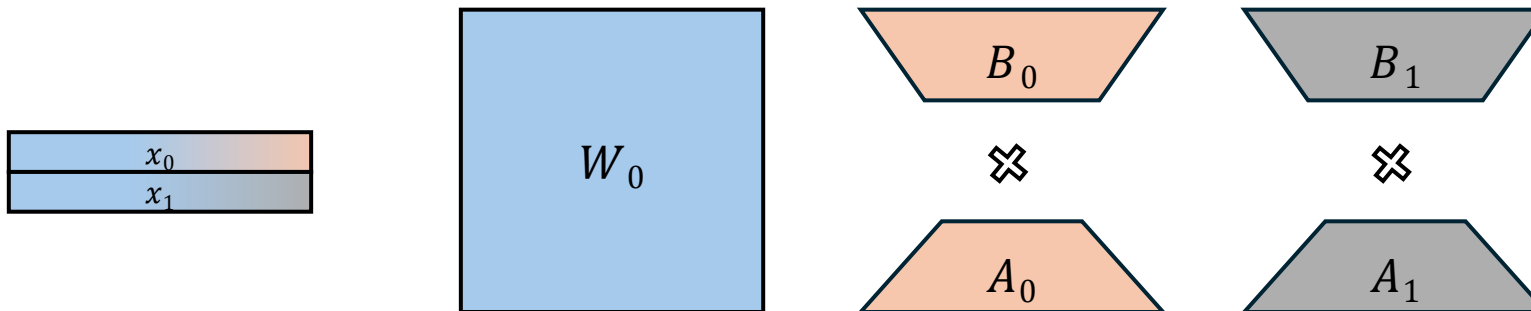
- Consider $x = [x_0, x_1]$, x_0 requires A_0, B_0 , x_1 requires A_1, B_1
- We want $y_0 = W_0x_0 + B_0A_0x_0$, $y_1 = W_0x_1 + B_1A_1x_1$
- Compute $W_0x = [W_0x_0, W_0x_1]$, $B_0A_0x_0$, $B_1A_1x_1$ separately



Merged vs. Unmerged Inference

Proposed: Unmerged Inference

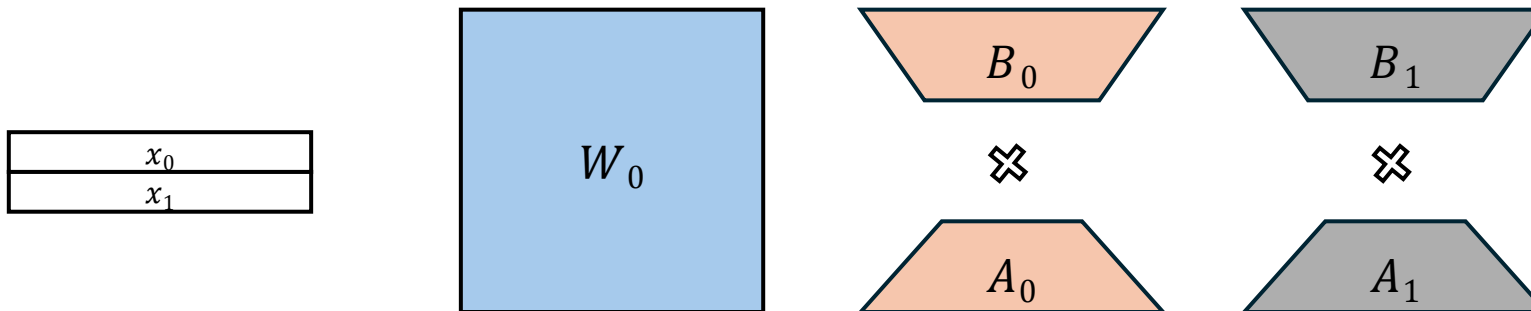
- Consider $x = [x_0, x_1]$, x_0 requires A_0, B_0 , x_1 requires A_1, B_1
- We want $y_0 = W_0x_0 + B_0A_0x_0$, $y_1 = W_0x_1 + B_1A_1x_1$
- Compute $W_0x = [W_0x_0, W_0x_1]$, $B_0A_0x_0$, $B_1A_1x_1$ separately
 - **Pros:** W_0x is more efficient on GPU due to larger matrix sizes (larger BS), and no need to do adaptor merge between requests
 - **Cons:** More flops compared to merged inference



Merged vs. Unmerged Inference

Proposed: Unmerged Inference

- Consider $x = [x_0, x_1]$, x_0 requires A_0, B_0 , x_1 requires A_1, B_1
- We want $y_0 = W_0x_0 + B_0A_0x_0$, $y_1 = W_0x_1 + B_1A_1x_1$
- Compute $W_0x = [W_0x_0, W_0x_1]$, $B_0A_0x_0, B_1A_1x_1$ separately
 - **Pros:** W_0x is more efficient on GPU due to larger matrix sizes (larger BS), and no need to do adaptor merge between requests
 - **Cons:** More flops compared to merged inference



When to Apply Unmerged Inference

The intuition behind scheduling

- If requests are **skewed** towards a single LoRA adaptor, we should apply merged inference
 - An extreme case would be only 1 LoRA adaptor available
- Otherwise, we should use unmerged inference to **maximize BS**
 - W_0x benefits from larger x
 - Separately computing $B_nA_nx_n$ costs extra flops, but overall throughput is still better

The scheduling algorithm

1. Find the adaptor with most pending requests
2. If currently **unmerged** and # requests is **greater** than threshold α_{switch}
 - Change to merged inference, and execute the adaptor with most requests
3. Otherwise, if currently **merged** and # requests is **less** than threshold β_{switch}
 - Change to unmerged inference

Additional Optimizations

Dynamic Threshold Tuning

- **How to decide the aforementioned threshold α and β**
- Employ an RL-like approach (pure empirical)
 - If in previous epoch, the algorithm select **merged inference**
 - Estimate the performance of merged/unmerged inference
 - If merged performance is indeed better
 - Change threshold so that heuristics is more likely to pick merge (**reward**)
 - Otherwise, change threshold to make merge less likely (**penalty**)
 - Similar approach goes for unmerged inference

Starvation Prevention

- Serving the most popular LoRA adaptor might starve others
- **Credit based** starvation prevention

Overall System Design

Intra-replica Scheduling

- Discussed in previous slides

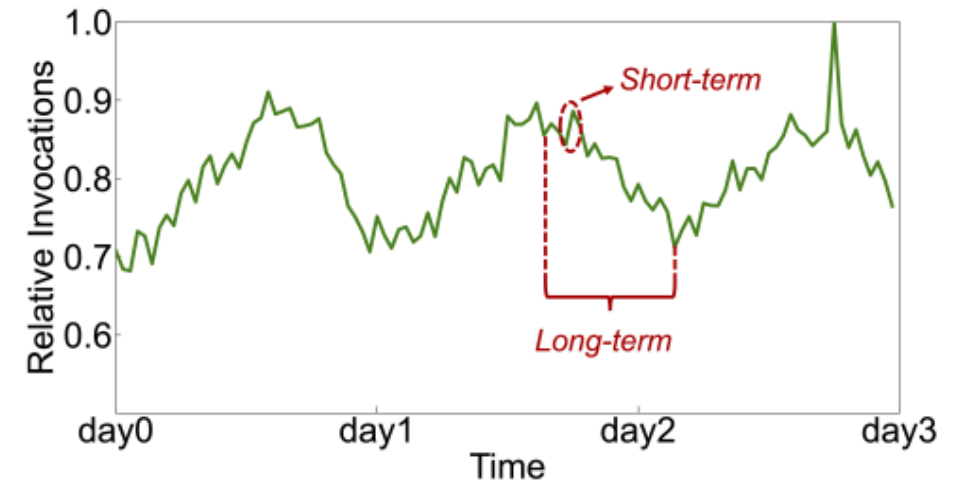
Inter-replica Scheduling

- **Proactive Mechanism**
 - Dispatch requests to replicas
- **Reactive Mechanism**
 - Monitor if imbalance happens
 - Trigger migration of adaptors across replicas

System-wide Load Balancing

Proactive Mechanism

- **Short term:** Dispatch to replica with lowest latency
 - This includes adaptor loading time if needed
 - Handles bursty load well
- **Long term:** Preload LoRA
 - **Goal:** maximize the minimum burst tolerance
 - Essentially making sure the system can handle burst load to any adaptor
 - **The tradeoff:**
 - More LoRA loaded enables the replica to handle more types of requests
 - LoRA also takes up GPU memory → Less memory for intermediate states (e.g., KV Cache)



System-wide Load Balancing

Reactive Mechanism

- **Proactive scheduling** alone is not enough due to **variable input/output length**
 - This still cause load imbalance across replicas in runtime
- **ILP Problem Formulation**
 - Given a set of requests and their adaptors, also a set of replicas
 - **Optimization goal:** minimizing global latency
 - **Constraint:**
 1. Request and corresponding adaptor must co-exist on same replica
 2. Replica memory consumption under capacity
 3. Each request is placed on only one replica
 - Apply ILP solver to find the solution
 - To reduce complexity, several constraints are relaxed

Evaluation

Implementation

- ~6.2k LoC on vLLM

Experiment Setup

- 4 x 8 A800 servers
- **Models:** Llama-2 family of models
- Trace sampled from ShareGPT and Azure
- **Metrics:** Avg latency, throughput under 0.5s SLO
- **Baseline:**
 - **vLLM:** Treat each LoRA LLM individually and equally
 - **PEFT:** Merged inference only, swap LoRA between batches

Strengths-dLoRA design

- Targets existing problem
- Correctly identifies that naive solutions (merge only, unmerge only, proactive dispatch) are wasteful
- Identifies the core tradeoffs (per-token compute vs. extra adapter compute)
- Correctly addresses fairness risk with starvation prevention
- Clearly defined ILP with pragmatic heuristics

Strengths-dLoRA experiments

- Realistic testbed: 32× A800 80GB GPUs
- Comparing throughput under latency SLO-aligns with dLoRA objectives
- Built on top of vLLM

Weaknesses-dLoRA

- Switching policy (between merged and unmerged inference) depends on thresholds (α , β) and runtime tuning. Stability behavior under rapidly shifting workloads isn't deeply analyzed.
- Credit-based starvation prevention is also heuristic.
- Practical, but lack supporting theories. E.g. convergence and worst-case bounds
- Performance depends on hyperparameter tuning for different systems.

Weaknesses-dLoRA

- Adapter-request co-migration: migrate LoRA adapters and requests (with intermediate states) from overloaded replicas to others.
- Requires moving KV cache, decode state, and LoRA adapter
- Increases operational complexity
- Migration complexity is arguably underexplored. E.g. realistic communication issues like fault tolerance

Weaknesses-dLoRA

- Adapter rank is fixed at $r = 8$
- Only tested on Llama-2 architecture
- Adapter ranks affects compute and memory, which is unexplored
- Different architectures can also affect performance
- Changing any of these will require careful hyperparameter tuning

Future directions

- Learning-based scheduler to replace manual hyperparameter tuning
- Consider realistic request distributions, such as Zipf popularity distributions
- Measure performance across a range of adapter ranks